

Guía de Actividades Práctico-Experimentales Nro. 008

1. Datos Generales

Asignatura	Estructura de Datos
Ciclo	3ro A / B
Unidad	2: Estructuras de Datos Lineales y Análisis Estocástico
Resultado de aprendizaje de la unidad	Desarrolla algoritmos basados en estructuras lineales tipo Pila (LIFO), asegurando la integridad transaccional mediante mecanismos de retroceso (backtracking) y control de estados.
Práctica Nro.	008
Título de la Práctica	Ordenación Verde (Divide y Vencerás) y Análisis de Búsqueda Secuencial vs. Binaria en Registros Indexados
Nombre del Docente	Cristian Ramiro Narváez Guillén
Fecha	jueves 17 y viernes 18 de Junio 2026
Horario	Jueves 28 de 10:30 – 11:30 Viernes 29 de mayo 11h30 – 13h30
Lugar	EVA Aula
Tiempo planificado en el Sílabo	3 horas

LINKZIP:

https://drive.google.com/drive/folders/1W8TMj5YV08lxo-__ZGFoaViKCxGjTz-M?usp=sharing

2. Objetivo(s) de la Práctica:

- Diseñar e implementar el algoritmo Merge Sort (o Quick Sort) aplicando el paradigma de "Divide y Vencerás" para ordenar una estructura de "Registros Ambientales" (Ordenación Verde) basados en índices numéricos y métricas.
- Codificar estrategias de Búsqueda Secuencial y Búsqueda Binaria para localizar registros específicos dentro del conjunto de datos.



unl

Universidad
Nacional
de Loja

- Evaluar y comparar empíricamente el tiempo computacional y la complejidad asintótica (vs) de los métodos de búsqueda en arreglos no ordenados vs. ordenados indexados, utilizando diferentes volúmenes de datos.

3. Materiales y reactivos:

- Hardware: Computadora personal con procesador 4 núcleos y 8GB RAM.
- Software: Java Development Kit (JDK) 17+, IDE (IntelliJ IDEA, Eclipse o NetBeans).
- Conectividad: Acceso a Internet para consulta de documentación técnica.
- Recursos Digitales: Sílabo de la asignatura, repositorio base en GitHub.
- Plataforma EVA UNL.

4. Equipos y herramientas

- Laboratorio de Computación Aplicada (equipado con conexión a internet).
- Computadora personal (PC o portátil) 4GB RAM.
- Software de procesamiento de textos (Microsoft Word / LibreOffice).
- EVA (Entorno Virtual de Aprendizaje UNL).
- Google Colab (Python 3x) IDE: Jupyter Notebook, VS Code.
- Navegador Web actualizado.
- Acceso a bibliografía digital o física.

5. Procedimiento / Metodología

La práctica emplea el Aprendizaje Basado en Investigación (ABI) para sustentar matemáticamente las complejidades temporales, y el Aprendizaje Basado en Problemas (ABP) para resolver la optimización de consultas dentro de un proyecto de reforestación (Ordenación Verde).

Tarea 1: Fundamentación Teórica y Análisis Asintótico (ABI)

1. **Investigue el Teorema Maestro (Master Theorem) y defina la relación de recurrencia que describe el algoritmo de ordenamiento Merge Sort o Quick Sort en su mejor caso y en el caso promedio.**

El Método Maestro.

¿Qué es?

El método maestro es una herramienta que sirve para calcular, de forma rápida y ordenada, cuánto tiempo le toma a un algoritmo resolver un problema cuando este se basa en la estrategia de "divide y vencerás". Es decir, cuando un problema grande se va partiendo en problemas más

pequeños hasta que se vuelven fáciles de resolver, y luego esas soluciones pequeñas se combinan para armar la solución final.

En vez de calcular ese tiempo paso a paso cada vez, el método maestro nos da una especie de fórmula general que ya tiene en cuenta las partes más importantes del proceso: cuántas veces se divide el problema, qué tan pequeño se vuelve cada parte, y cuánto cuesta unir todo al final.

La fórmula general

La recurrencia que el método maestro analiza tiene esta forma:

$$T(n) = aT(n/b) + f(n), \text{ donde } a \geq 1 \text{ y } b > 1$$

Cada parte de la fórmula representa algo concreto:

- **n**: el tamaño del problema original (por ejemplo, cuántos elementos tiene una lista que queremos ordenar).
- **a**: cuántas veces se repite el proceso de manera recursiva, es decir, en cuántas partes se divide el problema.
- **n/b**: el tamaño que le queda a cada una de esas partes después de dividir. Se asume que todas las partes quedan del mismo tamaño.
- **f(n)**: el costo de todo el trabajo que se hace fuera de las llamadas recursivas, como dividir el problema en partes y luego volver a juntar esas partes para dar la solución final.

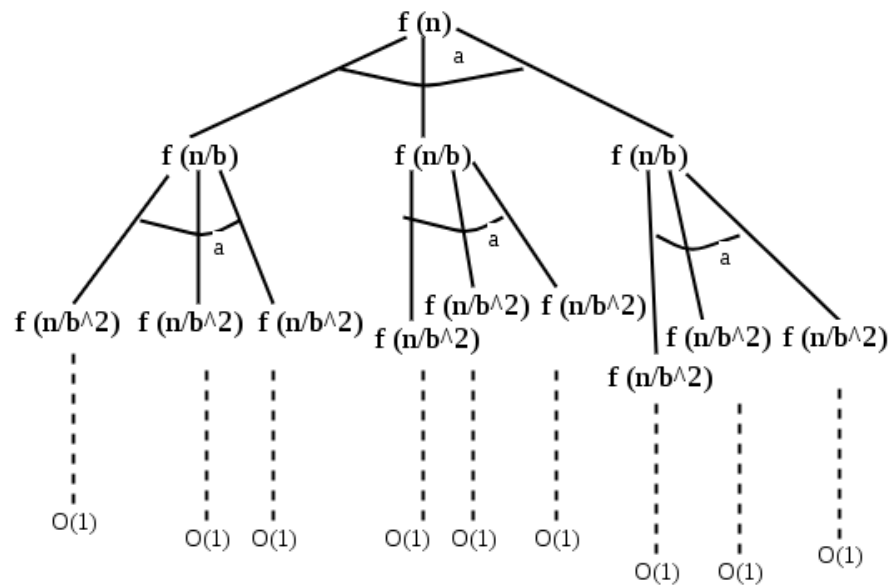
Los tres casos del método maestro

Dependiendo de cómo se comporte $f(n)$ en comparación con el resto de la fórmula, podemos estar en uno de tres escenarios:

1. Si el trabajo de combinar las partes ($f(n)$) crece más despacio que el trabajo de las llamadas recursivas, entonces el tiempo total queda dominado por las llamadas recursivas.
2. Si el trabajo de combinar las partes crece más o menos al mismo ritmo que las llamadas recursivas, el tiempo total termina siendo ese trabajo multiplicado por la cantidad de niveles que tiene el proceso.
3. Si el trabajo de combinar las partes crece más rápido que las llamadas recursivas, entonces el tiempo total queda dominado por ese trabajo de combinación.

¿Cómo se entiende esto de manera visual?

Una forma sencilla de entender el método es pensando en un árbol: en la parte de arriba (la raíz) está el costo de dividir y combinar el problema, representado por $f(n)$. En las puntas de las ramas (las hojas) está el trabajo más pequeño que ya no se puede dividir más. La altura de ese árbol depende de cuántas veces se puede seguir dividiendo el problema.



La imagen anterior representa el árbol de recurrencia generado por la fórmula $T(n) = aT(n/b) + f(n)$. En la parte superior (raíz) se encuentra el costo $f(n)$, correspondiente al trabajo de dividir el problema y combinar resultados. A medida que se desciende por el árbol, el tamaño del problema se reduce progresivamente (n/b , n/b^2 , etc.), y cada nivel se ramifica en "a" subproblemas. En la parte inferior (las hojas), los problemas ya son tan pequeños que su resolución toma un tiempo constante, representado como $O(1)$. Sumando el costo de todos los niveles del árbol se obtiene el tiempo total de ejecución, que es justamente lo que el método maestro permite calcular de forma directa.

Entonces, para saber el tiempo total que toma resolver todo, hay que ver qué parte "pesa más":

- Si el trabajo en las puntas del árbol es mayor, ese es el que manda.
- Si el trabajo está repartido más o menos parejo entre todos los niveles, se multiplica ese trabajo por la cantidad de niveles.
- Si el trabajo en la parte de arriba (la raíz) es mayor, entonces ese es el que define el resultado final.

¿Cuándo se puede usar este método?

El método maestro funciona bien cuando el problema se comporta de manera "ordenada", es decir, cuando se puede saber con claridad si el trabajo crece, se mantiene igual o disminuye conforme avanza la recursión. En general, sirve cuando:

- El trabajo necesario para resolver cada subproblema aumenta de forma constante en cada nivel.
- El trabajo necesario se mantiene prácticamente igual en todos los niveles.

- El trabajo necesario disminuye de forma constante en cada nivel.

¿Cuándo NO se puede usar?

Este método tiene sus límites. No se puede aplicar cuando:

2. La función $T(n)$ no se comporta de manera predecible o creciente (por ejemplo, si involucra una función como el seno, que sube y baja todo el tiempo).
3. La función $f(n)$ no tiene una forma matemática simple (como un polinomio), por lo que no encaja en el formato que el método necesita para funcionar.

Aplicación práctica: Merge Sort y Quick Sort

Tanto Merge Sort como Quick Sort son algoritmos de ordenamiento que también usan la estrategia de divide y vencerás, así que ambos se pueden analizar con el método maestro. Sin embargo, cada uno divide los datos de forma distinta, lo cual cambia un poco su relación de recurrencia.

Merge Sort

En este algoritmo, la forma de dividir los datos es siempre la misma, sin importar si estamos en el mejor caso o en el caso promedio. La recurrencia queda así:

$$T(n) = 2T(n/2) + \Theta(n)$$

- La parte **$2T(n/2)$** representa las dos llamadas recursivas que se hacen, ya que el problema se divide siempre exactamente a la mitad.
- La parte **$\Theta(n)$** representa el tiempo que toma volver a juntar las dos mitades ya ordenadas en una sola lista final.

Quick Sort

Acá la situación cambia dependiendo de qué tan bien (o mal) el "pivote" divide el arreglo:

- **Mejor caso:** ocurre cuando el pivote divide el arreglo justo por la mitad, dando como resultado la misma recurrencia que en Merge Sort:

$$T(n) = 2T(n/2) + \Theta(n)$$

- **Caso promedio:** ocurre cuando el pivote no divide el arreglo exactamente por la mitad, sino en proporciones distintas (por ejemplo, una parte mucho más grande que la otra), pero que en general siguen siendo proporcionales al tamaño total. En este caso, la recurrencia se expresa así: $T(n) = T(n \cdot a) + T(n(1-a)) + \Theta(n)$

4. La ecuación de recurrencia típica para Merge Sort es:

La ecuación es:

$$T(n) = 2T(n/2) + \Theta(n)$$

El $2T(n/2)$ representa las dos llamadas recursivas que se generan al dividir siempre el arreglo exactamente a la mitad, mientras que $\Theta(n)$ representa el costo de combinar las dos mitades ya ordenadas en una sola lista. Esta recurrencia es válida tanto para el mejor caso como para el caso promedio, ya que Merge Sort siempre divide de la misma manera sin importar el orden inicial de los datos.

5. Revise la precondition matemática fundamental que exige la Búsqueda Binaria para garantizar su correctitud lógica al dividir el espacio de búsqueda.

La Búsqueda Binaria exige que el arreglo o estructura sobre la que opera esté **ordenada** previamente, ordenada de forma consistente con el criterio de comparación que emplea el algoritmo. Esta condición es indispensable porque el algoritmo decide en cada paso descartar la mitad del espacio de búsqueda comparando el elemento central con el valor buscado: si el elemento central es menor que el objetivo, se descarta la mitad izquierda; si es mayor, se descarta la mitad derecha. Esta lógica sólo es válida bajo la garantía del orden, pues en un arreglo desordenado no existe ninguna relación predecible entre la posición de un elemento y su valor, lo que hace imposible saber en qué mitad continuar la búsqueda. En consecuencia, aplicar Búsqueda Binaria sobre datos no ordenados puede llevar a descartar la mitad donde sí está el elemento buscado, produciendo resultados incorrectos incluso cuando el elemento existe en el arreglo.

Tarea 2: Implementación de la "Ordenación Verde" (ABP)

Evolucione la gestión de datos creando una estructura orientada a registros de impacto ecológico (RegistroAmbiental). Codifique un algoritmo Divide y Vencerás (Merge Sort) en JavaScript para ordenar estos registros en base a su ID o su capacidad de absorción de CO₂.

// Estructura del Registro a ordenar y buscar

```
class RegistroAmbiental {  
  
  constructor(idRegistro, especie, toneladasCO2) {  
  
    this.idRegistro = idRegistro; // Índice principal  
  
    this.especie = especie;  
  
  }  
}
```



unl

Universidad
Nacional
de Loja

FEIRNNR - Carrera de Computación

```
this.toneladasCO2 = toneladasCO2;

}

}

// Implementación Parcial del TDA GestorAmbiental

class GestorRegistrosVerdes {

    // Tarea del estudiante: Implementar Merge Sort (Ordenación Verde)

    mergeSort(arreglo, inicio, fin) {

        if (inicio < fin) {

            let medio = Math.floor(inicio + (fin - inicio) / 2);

            // 1. Dividir (Llamadas recursivas)

            this.mergeSort(arreglo, inicio, medio);

            this.mergeSort(arreglo, medio + 1, fin);

            // 2. Vencer y Combinar

            this.merge(arreglo, inicio, medio, fin);

        }

    }

    merge(arreglo, inicio, medio, fin) {

        // Tarea del estudiante: Lógica de mezcla de los sub-arreglos ordenados

        // Priorizar el ordenamiento por 'idRegistro'

    }

}
```



unl

Universidad
Nacional
de Loja

FEIRNNR - Carrera de Computación

Tarea 3: Búsqueda Secuencial vs Binaria (ABP)

1. En la misma clase GestorRegistrosVerdes, implemente dos métodos de búsqueda que retornen el índice donde se encuentra un registro, o -1 si no existe:

- `busquedaSecuencial(arreglo, idBuscado)`
- `busquedaBinaria(arreglo, idBuscado)`

2. Genere estocásticamente tres escenarios de prueba (25,000, 500,000 y 1,000,000 de objetos RegistroAmbiental) utilizando el siguiente bloque de código. Asegúrese de comprender cómo el algoritmo de Fisher-Yates desordena el arreglo:

// Función para generar un arreglo de Registros Ambientales desordenados

```
function generarDatosPrueba(cantidad) {
```

```
  let datos = [];
```

```
  for (let i = 0; i < cantidad; i++) {
```

```
    // El idRegistro es secuencial temporalmente
```

```
    let especie = `Especie_${Math.floor(Math.random() * 100)}`;
```

```
    let co2 = +(Math.random() * 100).toFixed(2);
```

```
    datos.push(new RegistroAmbiental(i, especie, co2));
```

```
  }
```

```
  // Algoritmo de Fisher-Yates para desordenar el arreglo eficientemente en O(n)
```

```
  for (let i = datos.length - 1; i > 0; i--) {
```

```
    const j = Math.floor(Math.random() * (i + 1));
```

```
    [datos[i], datos[j]] = [datos[j], datos[i]];
```

```
  }
```

```
  return datos;
```

```
}
```



unl

Universidad
Nacional
de Loja

// Generación de los 3 escenarios

```
const dataset25K = generarDatosPrueba(25000);
```

```
const dataset500K = generarDatosPrueba(500000);
```

```
const dataset1M = generarDatosPrueba(1000000);
```

3. Mida el tiempo exacto que toma buscar el último ID posible de cada arreglo (es decir, 24999, 499999 y 999999) utilizando Búsqueda Secuencial en los arreglos originales (desordenados). Para medir el tiempo en milisegundos con alta precisión, utilice `performance.now()` como se ilustra en el siguiente ejemplo:

```
// Instanciar el gestor
```

```
const gestor = new GestorRegistrosVerdes();
```

```
// Capturar el tiempo justo antes de iniciar la búsqueda
```

```
const inicioTiempo = performance.now();
```

```
// Ejecutar la búsqueda (ejemplo con arreglo de 500k)
```

```
const resultado = gestor.búsquedaSecuencial(dataset500K, 499999);
```

```
// Capturar el tiempo justo al terminar la búsqueda
```

```
const finTiempo = performance.now();
```

```
const tiempoTotalMs = finTiempo - inicioTiempo;
```

```
console.log(`Búsqueda finalizada. Índice encontrado: ${resultado}`);
```

```
console.log(`Tiempo de ejecución: ${tiempoTotalMs.toFixed(4)} milisegundos`);
```

4. Aplique su método `mergeSort` a cada dataset y luego ejecute la **Búsqueda Binaria** para esos mismos IDs, utilizando la misma lógica de medición de tiempo. Se proporciona el siguiente código como base para que estructure sus pruebas comparativas:

```
// Ejemplo de medición para Búsqueda Binaria (dataset de 500k)
```

```
// 1. Primero, ordenamos el arreglo usando el método implementado
```

```
gestor.mergeSort(dataset500K, 0, dataset500K.length - 1);
```

```
// 2. Capturamos el tiempo de inicio
```

```
const inicioBinaria = performance.now();

// 3. Ejecutamos la búsqueda binaria

const resultadoBinaria = gestor.búsquedaBinaria(dataset500K, 499999);

// 4. Capturamos el tiempo de fin

const finBinaria = performance.now();

const tiempoBinariaMs = finBinaria - inicioBinaria;

console.log(`Búsqueda Binaria finalizada. Índice: ${resultadoBinaria}`);

console.log(`Tiempo de ejecución (Binaria): ${tiempoBinariaMs.toFixed(4)}
milisegundos`);
```

Compare el comportamiento y crecimiento de los tiempos obtenidos en los tres escenarios y redacte sus conclusiones en el informe.

6. Resultados esperados:

Al finalizar la práctica, el estudiante deberá entregar un informe técnico que incluya:

- **Estabilidad y Escalabilidad:** Un script en JavaScript completamente funcional capaz de ordenar un volumen masivo de datos (hasta 1,000,000 de registros) en pocos segundos, sin presentar desbordamientos en la pila de llamadas (*Stack Overflow*).
- **Validación Empírica:** Evidencia demostrable de que la *Búsqueda Binaria* localiza un registro en tiempo logarítmico (fracciones diminutas de milisegundo independientemente del tamaño), superando drásticamente el tiempo de ejecución lineal de la *Búsqueda Secuencial* conforme el tamaño del *dataset* escala.
- **Toma de Decisiones:** Comprensión analítica sobre el "costo inicial" de pre-ordenar los datos frente al "beneficio a largo plazo" al realizar múltiples búsquedas consecutivas sobre colecciones indexadas masivas.

Código clase `RegistroAmbiental`:

La clase `RegistroAmbiental` representa un registro de impacto ecológico. Su función es almacenar la información de cada elemento, incluyendo el identificador único (`idRegistro`), la especie registrada (`especie`) y la cantidad de dióxido de carbono absorbida (`toneladasCO2`). Actúa como la estructura de datos básica sobre la que se realizan los procesos de ordenamiento y búsqueda.

```
1
2
3 class RegistroAmbiental {
4
5     constructor (idRegistro, especie, toneladasCO2){
6         this.idRegistro = idRegistro
7         this.especie = especie
8         this.toneladasCO2 = toneladasCO2
9     }
10 }
11 module.exports = RegistroAmbiental;
```

Código GestorRegistros Verdes

La clase GestorRegistrosVerdes implementa las operaciones necesarias para gestionar los registros ambientales. Contiene los algoritmos de búsqueda secuencial y búsqueda binaria para localizar registros por su identificador, así como la implementación del algoritmo Merge Sort para ordenar los registros por idRegistro o por toneladasCO2. Su propósito es permitir el análisis y comparación del rendimiento de diferentes técnicas de búsqueda y ordenamiento.

```
2
3 class GestorRegistrosVerdes{
4
5     constructor(){
6     }
7
8     busquedaBinaria(arreglo, idBuscado) {
9         let inicio = 0;
10        let fin = arreglo.length - 1;
11
12        while (inicio <= fin) {
13            let medio = Math.floor((inicio + (fin - inicio) / 2));
14
15            if (arreglo[medio].idRegistro === idBuscado) {
16                return medio;
17            } else if (arreglo[medio].idRegistro < idBuscado) {
18                inicio = medio + 1;
19            } else {
20                fin = medio - 1;
21            }
22        }
23        return -1;
24    }
25
26    busquedaSecuencial(arreglo, idBuscado) {
27        for (let i = 0; i < arreglo.length; i++) {
28            if (arreglo[i].idRegistro === idBuscado) {
29                return i;
30            }
31        }
32        return -1;
33    }
34
35    mergeSort(arreglo, inicio, fin) {
36        if (inicio < fin) {
37
38            let medio = Math.floor((inicio + (fin - inicio) / 2));
39
40            this.mergeSort(arreglo, inicio, medio);
41            this.mergeSort(arreglo, medio + 1, fin);
42
43            this.merge(arreglo, inicio, medio, fin);
44        }
45    }
46
47    merge(arreglo, inicio, medio, fin) {
48
49        let izquierda = arreglo.slice(inicio, medio + 1);
50        let derecha = arreglo.slice(medio + 1, fin + 1);
51
52        let i = 0;
53        let j = 0;
54        let k = inicio;
55
56        while (i < izquierda.length && j < derecha.length) {
57
58            if (izquierda[i].idRegistro <= derecha[j].idRegistro) {
59                arreglo[k] = izquierda[i];
60            } else {
61                arreglo[k] = derecha[j];
62            }
63            k++;
64            if (i < izquierda.length) i++;
65            if (j < derecha.length) j++;
66        }
67
68        while (i < izquierda.length) {
69            arreglo[k] = izquierda[i];
70            k++;
71            i++;
72        }
73
74        while (j < derecha.length) {
75            arreglo[k] = derecha[j];
76            k++;
77            j++;
78        }
79    }
80 }
```



1859

```
3 class GestorRegistrosVerdes{
47   merge(arreglo, inicio, medio, fin) {
55
56       while (i < izquierda.length && j < derecha.length) {
57
58           if (izquierda[i].idRegistro <= derecha[j].idRegistro) {
59               arreglo[k] = izquierda[i];
60               i++;
61           } else {
62               arreglo[k] = derecha[j];
63               j++;
64           }
65
66           k++;
67       }
68
69       while (i < izquierda.length) {
70           arreglo[k] = izquierda[i];
71           i++;
72           k++;
73       }
74
75       while (j < derecha.length) {
76           arreglo[k] = derecha[j];
77           j++;
78           k++;
79       }
80   }
81
82   mergeSortPorCO2(arreglo, inicio, fin) {
83
84       if (inicio < fin) {
85
86           let medio = Math.floor(inicio + (fin - inicio) / 2);
87
88           this.mergeSortPorCO2(arreglo, inicio, medio);
89           this.mergeSortPorCO2(arreglo, medio + 1, fin);
90
91           this.mergePorCO2(arreglo, inicio, medio, fin);
92       }
93   }
94
95   mergePorCO2(arreglo, inicio, medio, fin) {
96
97       let izquierda = arreglo.slice(inicio, medio + 1);
98       let derecha = arreglo.slice(medio + 1, fin + 1);
99
100       let i = 0;
101       let j = 0;
102       let k = inicio;
103
104       while (i < izquierda.length && j < derecha.length) {
105
106           if (izquierda[i].toneladasCO2 <= derecha[j].toneladasCO2) {
107               arreglo[k] = izquierda[i];
108               i++;
109           } else {
110               arreglo[k] = derecha[j];
111               j++;
112           }
113       }
114       while (i < izquierda.length) {
115           arreglo[k] = izquierda[i];
116           i++;
117           k++;
118       }
119       while (j < derecha.length) {
120           arreglo[k] = derecha[j];
121           j++;
122           k++;
123       }
124   }
125 }
```

```
1859
82  class GestorRegistrosVerdes{
83      mergeSortPorCO2(arreglo, inicio, fin) {
84
85      }
86
87      this.mergeSortPorCO2(arreglo, inicio, medio);
88      this.mergeSortPorCO2(arreglo, medio + 1, fin);
89
90      this.mergePorCO2(arreglo, inicio, medio, fin);
91      }
92  }
93
94
95  mergePorCO2(arreglo, inicio, medio, fin) {
96
97      let izquierda = arreglo.slice(inicio, medio + 1);
98      let derecha = arreglo.slice(medio + 1, fin + 1);
99
100     let i = 0;
101     let j = 0;
102     let k = inicio;
103
104     while (i < izquierda.length && j < derecha.length) {
105
106         if (izquierda[i].toneladasCO2 <= derecha[j].toneladasCO2) {
107             arreglo[k] = izquierda[i];
108             i++;
109         } else {
110             arreglo[k] = derecha[j];
111             j++;
112         }
113
114         k++;
115     }
116
117     while (i < izquierda.length) {
118         arreglo[k] = izquierda[i];
119         i++;
120         k++;
121     }
122
123     while (j < derecha.length) {
124         arreglo[k] = derecha[j];
125         j++;
126         k++;
127     }
128 }
129 }
130 module.exports = GestorRegistrosVerdes;
```

Consola Prueba Resultados de Medición:

El archivo **Main.js** actúa como punto de entrada de la aplicación. Su función es generar los conjuntos de datos de prueba utilizando registros ambientales aleatorios, instanciar el gestor de registros, ejecutar los algoritmos de búsqueda secuencial y búsqueda binaria, aplicar el algoritmo Merge Sort para ordenar los datos y medir los tiempos de ejecución mediante **performance.now()**. Además, muestra los resultados obtenidos para comparar el rendimiento de las diferentes técnicas de búsqueda en distintos tamaños de datos.

```
1  const RegistroAmbiental = require("./RegistroAmbiental");
2  const GestorRegistrosVerdes = require("./GestorRegistrosVerdes");
3  const { performance } = require("perf_hooks");
4  function generarDatosPrueba(cantidad) {
5      let datos = [];
6
7      for (let i = 0; i < cantidad; i++) {
8          let especie = `Especie_${Math.floor(Math.random() * 100)}`;
9          let co2 = +(Math.random() * 100).toFixed(2);
10
11         datos.push(new RegistroAmbiental(i, especie, co2));
12     }
13
14     // Fisher-Yates
15     for (let i = datos.length - 1; i > 0; i--) {
16         const j = Math.floor(Math.random() * (i + 1));
17         [datos[i], datos[j]] = [datos[j], datos[i]];
18     }
19
20     return datos;
21 }
22
23 const gestor = new GestorRegistrosVerdes();
24
25 const dataset25K = generarDatosPrueba(25000);
26 const dataset500K = generarDatosPrueba(500000);
27 const dataset1M = generarDatosPrueba(1000000);
28
29 console.log("=== BUSQUEDA SECUENCIAL ===");
30
31 let inicio = performance.now();
32 let resultado = gestor.busquedaSecuencial(dataset25K, 24999);
33 let fin = performance.now();
34
35 console.log("Indice:", resultado);
36 console.log("Tiempo:", (fin - inicio).toFixed(4), "ms");
37
38 inicio = performance.now();
39 resultado = gestor.busquedaSecuencial(dataset500K, 499999);
40 fin = performance.now();
41
42 console.log("Indice:", resultado);
43 console.log("Tiempo:", (fin - inicio).toFixed(4), "ms");
44
45 inicio = performance.now();
46 resultado = gestor.busquedaSecuencial(dataset1M, 999999);
47 fin = performance.now();
```

```
22
23 const gestor = new GestorRegistrosVerdes();
24
25 const dataset25K = generarDatosPrueba(25000);
26 const dataset500K = generarDatosPrueba(500000);
27 const dataset1M = generarDatosPrueba(1000000);
28
29 console.log("=== BUSQUEDA SECUENCIAL ===");
30
31 let inicio = performance.now();
32 let resultado = gestor.busquedaSecuencial(dataset25K, 24999);
33 let fin = performance.now();
34
35 console.log("Indice:", resultado);
36 console.log("Tiempo:", (fin - inicio).toFixed(4), "ms");
37
38 inicio = performance.now();
39 resultado = gestor.busquedaSecuencial(dataset500K, 499999);
40 fin = performance.now();
41
42 console.log("Indice:", resultado);
43 console.log("Tiempo:", (fin - inicio).toFixed(4), "ms");
44
45 inicio = performance.now();
46 resultado = gestor.busquedaSecuencial(dataset1M, 999999);
47 fin = performance.now();
48
49 console.log("Indice:", resultado);
50 console.log("Tiempo:", (fin - inicio).toFixed(4), "ms");
51
52 console.log("\n=== ORDENANDO CON MERGE SORT ===");
53
54 gestor.mergeSort(dataset25K, 0, dataset25K.length - 1);
55 gestor.mergeSort(dataset500K, 0, dataset500K.length - 1);
56 gestor.mergeSort(dataset1M, 0, dataset1M.length - 1);
57
58 console.log("\n=== BUSQUEDA BINARIA ===");
59
60 inicio = performance.now();
61 resultado = gestor.busquedaBinaria(dataset25K, 24999);
62 fin = performance.now();
63
64 console.log("Indice:", resultado);
65 console.log("Tiempo:", (fin - inicio).toFixed(4), "ms");
66
67 inicio = performance.now();
68 resultado = gestor.busquedaBinaria(dataset500K, 499999);
69 fin = performance.now();
70
71 console.log("Indice:", resultado);
72 console.log("Tiempo:", (fin - inicio).toFixed(4), "ms");
73
74 inicio = performance.now();
75 resultado = gestor.busquedaBinaria(dataset1M, 999999);
76 fin = performance.now();
77
78 console.log("Indice:", resultado);
79 console.log("Tiempo:", (fin - inicio).toFixed(4), "ms");
```

```
• kleynerbuzz@kleynerbuzz-ASUS-TUF-Gaming-F15-FX507ZU4-FX507ZU4:~/Documentos/Estructura_Datos/Estructura_Datos/StructureData/Unidad02/practica8$ node
=== BUSQUEDA SECUENCIAL ===
Indice: 2587
Tiempo: 0.3968 ms
Indice: 441488
Tiempo: 7.5076 ms
Indice: 423232
Tiempo: 8.9085 ms

=== ORDENANDO CON MERGE SORT ===

=== BUSQUEDA BINARIA ===
Indice: 24999
Tiempo: 0.1664 ms
Indice: 499999
Tiempo: 0.0261 ms
Indice: 999999
Tiempo: 0.0085 ms
• kleynerbuzz@kleynerbuzz-ASUS-TUF-Gaming-F15-FX507ZU4-FX507ZU4:~/Documentos/Estructura_Datos/Estructura_Datos/StructureData/Unidad02/practica8$
```

7. Preguntas de Control:

- Con base en la ejecución empírica de la Tarea 3, ¿por qué la **Búsqueda Binaria** presenta una mejora de rendimiento tan drástica en comparación a la **Búsqueda Secuencial**, expresado en términos de iteraciones máximas necesarias para ?

La drástica mejora de rendimiento radica en que la búsqueda secuencial opera con una complejidad lineal $\Theta(n)$, obligando al algoritmo a examinar hasta el total de los elementos (1,000,000 de iteraciones en el peor caso para el *dataset1M*), mientras que la búsqueda binaria utiliza la estrategia de Divide y Vencerás con una complejidad logarítmica $\Theta(\log_2 n)$. Al dividir el espacio de búsqueda exactamente a la mitad en cada paso, el número máximo de iteraciones necesarias para reducir cualquier problema a la unidad se rige por la ecuación $2^I = n \Rightarrow I = \lceil \log_2 n \rceil$, lo que se traduce matemáticamente en un máximo de apenas 10 iteraciones para 1,000 registros y tan solo 20 iteraciones para 1,000,000 de registros, logrando una eficiencia computacional casi instantánea.

1. Si un arreglo ya se encuentra casi ordenado (por ejemplo, con solo un 5% de registros fuera de lugar), ¿qué algoritmo de ordenamiento sería teóricamente más veloz, **Merge Sort** o **Insertion Sort**? Sustente asintóticamente su respuesta.

Para un arreglo con el 95% de sus elementos ya en la posición correcta, **Insertion Sort** será teóricamente más veloz que Merge Sort debido a que su complejidad temporal en el mejor caso es lineal $\Omega(n)$ y se rompe el ciclo interno de comparación de forma constante $O(1)$ cuando los elementos ya están ordenados. Por el contrario, Merge Sort es completamente insensible al orden inicial de los datos y mantendrá de forma estricta una complejidad de $\Theta(n \log_2 n)$ en todos sus escenarios, ya que obligatoriamente ejecutará el árbol completo de divisiones y fusiones recursivas. Al estar el arreglo casi ordenado, Insertion Sort se aproxima a un comportamiento asintótico lineal **$O(n)$** , ahorrando además la costosa sobrecarga de llamadas en la pila y la asignación de memoria que Merge Sort requiere.

2. Describa por qué el peor caso computacional del algoritmo **Quick Sort** degrada a y mencione qué estrategia de pivote (Divide y Vencerás) puede mitigar este riesgo.

El peor caso de Quick Sort degrada a una complejidad temporal de $O(n^2)$ cuando el pivote elegido en cada paso resulta ser sistemáticamente el elemento mínimo o máximo del subarreglo (común al procesar datos ya ordenados secuencialmente con un pivote fijo en los extremos), lo que desbalancea por completo las particiones en tamaños de 0 y $n - 1$. Esta asimetría transforma el árbol de recursión balanceado de altura $\log n$ en una estructura lineal de altura n cuyo trabajo total equivale a la suma de una

serie aritmética, pero este riesgo se mitiga aplicando la estrategia de **Pivote de la Mediana de Tres** o mediante un **Pivote Aleatorio**, técnicas de *Divide y Vencerás* que garantizan probabilísticamente particiones equilibradas y devuelven al algoritmo a su rendimiento óptimo de $O(n \log n)$.

3. ¿Es posible realizar una Búsqueda Binaria sobre una estructura enlazada tradicional sin índices directos conservando su eficiencia teórica de ? Explique detallando el costo de acceso a memoria y la naturaleza de las referencias en JavaScript.

No es posible conservar la eficiencia teórica de $O(\log n)$ en una lista enlazada tradicional debido a que la búsqueda binaria exige un acceso aleatorio e inmediato de costo $O(1)$ al elemento central ($\text{medio} = (\text{inicio} + \text{fin}) / 2$), una propiedad física que estas estructuras no poseen al carecer de índices directos. En su lugar, la complejidad se degrada inevitablemente a $O(n)$ porque para hallar el nodo central el algoritmo se ve obligado a recorrer secuencialmente los punteros intermedios (nodo.next), sumado a que en el entorno de JavaScript los objetos se alojan dispersos en el *Heap* de memoria, lo que destruye la localidad de los datos, genera constantes fallos de caché (*Cache Misses*) en la CPU y penaliza críticamente el rendimiento frente a la lectura contigua de un arreglo estándar.

4. Analice la complejidad espacial de su implementación de Merge Sort. ¿Por qué se considera que este algoritmo requiere de memoria auxiliar y cómo podría impactar esto al ordenar volúmenes inmensos de datos en el entorno de ejecución del motor V8 de JavaScript (Node.js o Navegador)?

La complejidad espacial de Merge Sort es $O(n)$ debido a que no es un algoritmo *in-place* y requiere obligatoriamente alojar arreglos dinámicos auxiliares para duplicar y preservar temporalmente las sublistas izquierda y derecha durante la fase de combinación (merge) sin sobrescribir datos activos. Al procesar volúmenes masivos de datos como el dataset1M, esta duplicación constante genera una severa presión sobre el *Garbage Collector* del motor V8 (Node.js o navegador), el cual debe pausar frecuentemente el hilo principal de ejecución (*Stop-the-world*) para recolectar millones de arreglos efímeros descartados, elevando críticamente la latencia del sistema o provocando directamente un colapso del programa por desbordamiento de memoria conocido como Heap Out of Memory.

Conclusiones

- Al implementar el ordenamiento por Merge Sort, queda claro que para manejar volúmenes grandes de registros ambientales no se puede depender de métodos improvisados, cuando los datos escalan a cientos de miles o un millón de filas, este método estructurado demuestra ser completamente necesario para mantener el programa estable y evitar que colapse o se congele.

- Existe una relación directa entre el tiempo que invertimos en ordenar la información y la velocidad para encontrarla, si los datos cambian todo el tiempo y solo necesitamos buscar algo una sola vez, la búsqueda secuencial es la mejor opción porque no perdemos tiempo reacomodando nada, sin embargo, si vamos a consultar la lista una y otra vez, vale la pena ordenar el arreglo al principio, ya que después la búsqueda binaria encuentra cualquier registro de forma casi instantánea, sin importar el tamaño del grupo.
- Procesar un volumen masivo de objetos en JavaScript exige un esfuerzo considerable tanto para el procesador como para la memoria RAM, al usar funciones recursivas que se llaman a sí mismas para dividir el problema, es fundamental programar de forma limpia y ordenada, de lo contrario, el motor del navegador o de Node.js se queda sin recursos en corto tiempo, afectando la estabilidad del sistema al gestionar estos datos ecológicos.
- La eficiencia de un algoritmo depende tanto de su lógica asintótica como de su interacción con el entorno de ejecución. Mientras que la búsqueda binaria y Quick Sort maximizan el rendimiento mediante la división equilibrada sobre arreglos indexados, Merge Sort revela un costo oculto de $O(n)$ en memoria auxiliar. En el motor V8 de JavaScript, esta duplicación de datos satura el **Garbage Collector** ante volúmenes masivos, demostrando que la elección del algoritmo ideal exige un equilibrio estricto entre la velocidad de la CPU, la estructura de datos y la gestión de recursos del sistema.

8. Evaluación

Se evaluará con base a la siguiente rúbrica.

Criterio Evaluado	Evidencia Esperada	Puntaje Máximo
Fundamentación Teórica	Análisis de la ecuación de recurrencia y resolución del Teorema Maestro en el reporte.	2.0
Algoritmo Divide y Vencerás	Implementación funcional de mergeSort y comprensión de su consumo de pila / memoria.	3.0
Mecanismos de Búsqueda	Implementación correcta de búsquedaSecuencial y búsquedaBinaria sobre la clase de registros.	3.0
Análisis y Conclusiones	Comparación estocástica objetiva (performance.now()) en los 3 tamaños solicitados.	2.0
TOTAL		10.0



9. Bibliografía

[1] GeeksforGeeks. "Master Theorem for Divide and Conquer Recurrences - GeeksforGeeks". GeeksforGeeks. Accedido el 25 de junio de 2026. [En línea]. Disponible:

<https://www.geeksforgeeks.org/dsa/advanced-master-theorem-for-divide-and-conquer-recurrences/>

[2] "Master Theorem (With Examples)". Programiz: Learn to Code for Free. Accedido el 25 de junio de 2026. [En línea]. Disponible:

<https://www.programiz.com/dsa/master-theorem>

[3] BlackeyeB. "Algoritmos de ordenación explicados con ejemplos en JavaScript, Python, Java y C++". freeCodeCamp.org. Accedido el 25 de junio de 2026. [En línea]. Disponible:

<https://www.freecodecamp.org/espanol/news/algoritmos-de-ordenacion-explicados-con-ejemplos-en-javascript-python-java-y-c/>

[4] K. Tewani. "What is the recurrence relation of the best case in Quicksort?" Quora. Accedido el 25 de junio de 2026. [En línea]. Disponible:

<https://www.quora.com/What-is-the-recurrence-relation-of-the-best-case-in-Quicksort>

[5] M. Taylor, "Writing correct code, part 3: preconditions and postconditions (binary search part 4c)," *The Reinvigorated Programmer*, Apr. 30, 2010. [En línea]. Disponible en:

<https://reprog.wordpress.com/2010/04/30/writing-correct-code-part-3-preconditions-and-postconditions-binary-search-part-4c/>



1. Elaboración y Aprobación

- Elaborado por: nombre del Docente responsable de la práctica.
- Aprobado por: nombre del Director de Carrera.

**Elaborado
por**

Cristian R. Narváez Guillén
Docente

**Aprobado
por**

Edison L Coronel Romero
Director de Carrera
